

S.N	TABLE OF CONTENT	DATE OF SUBMISSION
1.	Implementation of string operations.	2081/04/02
2.	Implementation of DFA	2081/04/16
3.	Implementation of NFA	2081/04/23
4.	checking keywords, Identifiers and the comments in C.	2081/06/04
5.	Implementation of Turing machine	2081/06/08
6.	Implementation of PDA	2081/06/08
7.		
8.		



TRIBHUVAN UNIVERSITY
INSTITUTE OF SCIENCE AND TECHNOLOGY



HIMALAYA COLLEGE OF ENGINEERING
CHYASAL, LALITPUR

Checked
EF

Lab Report No:- 1

Title:- Implementation of string operations

Submitted by:-

Name:- swajan shrestha

Roll No:- 38

Submitted To:- Hari prasad Pokharel

Department Of CSIT

Checked by:-

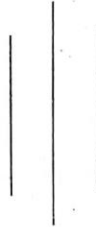
Date of submission:- 2081/04/02



TRIBHUVAN UNIVERSITY
INSTITUTE OF SCIENCE AND TECHNOLOGY



HIMALAYA COLLEGE OF ENGINEERING
CHYASAL, LALITPUR



Checked
[Signature]

Lab Report No:- 1

Title:- Implementation of string operations

Submitted by:-

Name:- swajan shrestha

Roll No:- 38

Submitted To:- Hari prasad Pokharel

Department Of CSIT

Checked by:-

Date of submission:- 2081/04/02

TITLE: Implementation of string operations.

OBJECTIVE:

→ The objective of this lab is to implement various string operations in C without using standard library functions. These operations include finding string length, reversing strings, concatenating strings, finding suffixes and the prefixes and extracting substrings.

Software used:

- visual studio code (vscode).
- language used = C / C++.

THEORY:

→ 1, finding string length: To find the length of a string 's' without using library functions, iterate through each character of string until the null character '\0' is encountered, counting each character.

2, Reversing a string: To reverse a string 's', iterate through the string from the end of the beginning and copy each character into a new string 's1' starting from the beginning.

3, Concatenating strings: Concatenating two strings 's1' and 's2' into 's3' involves iterating through 's1' to copy its characters into 's3', followed by iterating through 's2' to append its characters to 's3'.

4, finding suffixes: To find all suffixes of a string 's', iterate through 's' and print substrings starting from each index 'i' to the end of the string.

5, finding prefixes: To find all prefixes of a string 's', iterate through 's' and print substrings starting from the beginning upto each index 'i'.

6, Extracting substring: Extracting a substring 's1' from string 's' starting from index 'i' with length 'p' involves iterating through 's' ~~str~~ starting from index 'i' and copying 'p' characters into 's1'.

OBSERVATION :

a, Given a string's find it's length without using library function.

// SOURCE - code :

```
#include <stdio.h>
#include <conio.h>
int main ()
{
    char s[100];
    int i, length = 0;
    // Input string
    printf ("Enter a string :");
    gets(s);
    for (i=0; s[i] != '\0'; i++)
    {
        length = length + 1;
    }
    printf ("length of the string is : %d\n", length);
    getch();
    return 0;
}
```

INPUT :

Enter a string : swajan shrestha
length of the string is : 16

b, Given string 's' print's its reverse string s1.

// source code :

```
#include <stdio.h>
#include <conio.h>

int main()
{
    char s[100], s1[100];
    int i, length = 0;
    printf("Enter a string:");
    gets(s);
    for (i = 0; s[i] != '\0'; i++)
        length++;
    for (i = 0; i < length; i++)
        s1[i] = s[length - i - 1];
    s1[length] = '\0';
    printf("Reversed string is: %s\n", s1);
    getch();
    return 0;
}
```

OUTPUT :

Enter a string: swajan shrestha
Reversed string is: ahtsehts najarus

> Given two strings s1 and s2 displays the concatenated string s3 = s1.s2.

// source code:

```
#include <stdio.h>
#include <conio.h>

int main()
{
    char s1[100], s2[100], s3[200];
    int i, j;
    printf("Enter the first string:");
    gets(s1);
    printf("Enter the second string:");
    gets(s2);
    for (i=0; s1[i] != '\0'; i++)
    {
        s3[i] = s1[i];
    }
    for (j=0; s2[j] != '\0'; j++, i++)
    {
        s3[i] = s2[j];
    }
    s3[i] = '\0';
    printf("Concatenated string is: %s\n", s3);
    getch();
    return 0;
}
```

OUTPUT:

Enter first string: swajan

Enter second string: shrestha

Concatenated string is: swajanshrestha

Q> Given a string s, find all suffix. eg:
abcd, bcd, cd, d.

//source_code:

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
int main()
```

```
{ char s[100];
```

```
int p, length = 0;
```

```
printf("Enter a string:");
```

```
gets(s);
```

```
for (p = 0; s[p] != '\0'; p++)
```

```
{ length++; }
```

```
printf("All suffixes of string are:\n");
```

```
for (p = 0; p < length; p++)
```

```
{ printf("%s\n", s+p); }
```

```
getch();
```

```
return 0;
```

```
}
```

OUTPUT:

Enter a string: surajan

All suffixes of the string are:

surajan

urajan

rajan

ajan

jan

an

n

e> Given a string, s, find all prefix. eg: abc,
ab, a

// source - c++

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
int main()
```

```
{ char s[100];
```

```
  int i, j, length = 0;
```

```
  printf("Enter a string:");
```

```
  gets(s);
```

```
  for (i = 0; s[i] != '\0'; i++)
```

```
  { length ++; }
```

```
  printf("All prefixes of string are:\n");
```

```
  for (i = length; i > 0; i--)
```

```
  { for (j = 0; j < i; j++) }
```

```
    printf("%c", s[j]);
```

```
  }
```

```
  printf("\n");
```

```
}
```

```
getch();
```

```
return 0;
```

```
}
```

OUTPUT: Enter a string : hcoe

All prefixes of the string are:

hcoe

hco

hc

h

Given a string s , find its substring from index i to length p . Eg:- $s = 'abcde'$
substring $i = 1, p = 3$ $s1 = bcd$

// source_code:

```
#include <conio.h>
#include <stdio.h>
#include <string.h>
int main() {
    char s[100], s1[100];
    int i, p, j, length;
    printf("Enter the string: ");
    gets(s);
    length = strlen(s);
    printf("Enter the starting index: ");
    scanf("%d", &i);
    printf("Enter the length of substring: ");
    scanf("%d", &p);
    if (i < 0 || i >= length) {
        printf("Invalid starting index.\n");
    } else if (p < 0 || i + p > length) {
        printf("Invalid length for substring.\n");
    } else {
        for (j = 0; j < p && s[i+j] != '\0'; j++)
            s1[j] = s[i+j];
        s1[j] = '\0';
        printf("The substring is: %s\n", s1);
    }
    return 0; }
```

OUTPUT: Enter the string: surajan
Enter the starting index: 1
Enter the length of substring: 4
The substring is: uraj

DISCUSSION:

→ String operations in C are foundational for the manipulating text data, involving tasks such as determining string length, reversing strings, concatenating strings and extracting substrings. These operations enhance programming skills by reinforcing concepts like array handling, pointer manipulation and efficient memory management.

Understanding how to efficiently handle strings is crucial for developing robust software applications. Efficient string manipulation techniques enable developers to process user inputs, manage file contents, and implement data parsing algorithms effectively.

CONCLUSION:

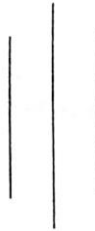
→ In conclusion, string operations in C plays a pivotal role in shaping competent programmers, equipping them with essential skills to tackle diverse programming tasks and contribute effectively to the development of the sophisticated software systems.



TRIBHUVAN UNIVERSITY
INSTITUTE OF SCIENCE AND TECHNOLOGY



HIMALAYA COLLEGE OF ENGINEERING
CHYASAL, LALITPUR



checked
[Signature]

Lab Report No:- 2

Title:- Implementation of DFA

Submitted by:-

Name:- swajan shrestha

Roll No:- 38

Submitted To:- Hari Prasad pokharel

Department Of CSIT

Checked by:-

Date of submission:- 2081/04/16

TITLE: Implementation of DFA

OBJECTIVE:-

- To understand the concept of Deterministic finite automata (DFA).
- To design & implement a DFA that accepts specific binary patterns.
- To simulate generic DFA's as defined by user inputs.

THEORY:-

1. Deterministic finite automation (DFA):

→ A DFA is a theoretical model of computation used in automata theory, computer science and formal language theory. It represents a finite state machine that accepts or rejects strings of symbols. The machine reads a string of symbols, one symbol at a time, and transitions between states according to a predefined set of rules.

The machine reads a string of symbols, one symbol at a time, and transitions between states according to a predefined set of rules. Each state represents a condition or configuration of the system at a particular point in time. The DFA is deterministic because for each state and the symbol there is exactly one transition to a next state.

2. Components of DFA:

- States (Q): A finite set of states the automaton can be in. It includes one start state and a set of accepting (or final) states.
- Alphabet (Σ): A finite set of symbols that the automaton reads. These symbols form the input strings processed by the DFA.
- Transition function (δ): A function that takes a state and an input symbol and returns a new state. formally, it is defined as $\delta: Q \times \Sigma \rightarrow Q$.
- Start State (q_0): The state at which the automaton begins processing the input string. It is one of the states in Q .
- Accepting states (F): A set of states that represent successful completion of the input string processing. If the DFA ends in one of these states after processing the entire input string, the string is accepted.

3. Working of DFA:

- The DFA starts in the initial state.
- It reads symbols from the input string one by one.
- For each symbol, the DFA uses the transition function to determine the next state from the current state.
- This process continues until all symbols in input

string are read.

- After reading the input string, if the DFA is in an accepting state, the input string is accepted; otherwise, it is rejected.

4, Designing a DFA:-

- a, Identify the pattern: Determine the specific pattern or set of patterns the DFA should recognize. For example, binary strings containing '01' or strings with substrings '1011'.
- b, Define states: Create states representing the different stages of processing the input string.
- c, Transition function: Define the transitions between states based on the input symbols. These transitions should capture the logic needed to recognize the desired pattern.
- d, Start and Accepting states: Specify the initial state and one or more accepting states where the DFA should end to accept the input string.

5, Application of DFA:-

- a, Text search Algorithms: DFA's are used to find patterns in strings efficiently. For instance, searching for a specific word in a text document.
- b, Lexical Analysis: In compiler design, DFAs

is used to tokenize input code, identifying the keywords, operators, identifiers and other components.

Network Protocol Analysis: DFAs help in analysing and verifying the correctness of network protocols by modeling the sequence of messages and states in the communication.

By understanding & implementing DFA's, we can gain insights into the mechanics of pattern recognition, state transitions and computational efficiency. These concepts form the basis for more advanced topics in computer science, such as parsing algorithms, regular expressions and formal language theory.

Programs and Implementations:-

a, DFA to Accept string with '01'. Design

→ #include <stdio.h>

```
enum states {q0, q1, qd, qf};  
enum states delta (enum states, char);
```

```
int main()
```

```
{ char input [20];
```

```
enum states curr_state = q0;
```

```
int i = 0;
```

```
printf("\n Enter a binary string:");
```

```
gets(input);
```

```
char ch = input[i];
```

```
while (ch != '\0') {
```

```
curr_state = delta (curr_state, ch);
```

```
ch = input[++i];
```

```
}
```



```

if (curr_state == qf) {
    printf("provided string is Accepted");
} else {
    printf("provided string is Rejected");
}
return 0;

```

```

}

```

```

enum states delta (enum states s, char ch)

```

```

{ enum states curr_state;

```

```

    switch (s)

```

```

    { case q0:

```

```

        if (ch == '0')

```

```

            curr_state = q1;

```

```

        else

```

```

            curr_state = qd;

```

```

        break;

```

```

    case q1:

```

```

        if (ch == '1')

```

```

            curr_state = qf;

```

```

        else

```

```

            curr_state = qd;

```

```

        break;

```

```

    case qf:

```

```

        curr_state = qf;

```

```

        break;

```

```

    case qd:

```

```

        curr_state = qd;

```

```

        break;

```

```

}

```

```

return curr_state;

```

```

}

```

OUTPUT:

Enter a binary string : 1001
provided string is Rejected.

Design a DFA that accepts the strings that contains '1011' as substrings. Also implement it using C.

```
→ #include <stdio.h>
enum states { q0, q1, q2, q3, qf, qd };
enum states delta( enum states, char );
int main()
{
    char input[100];
    enum states curr_state = q0;
    int i = 0;
    printf("\n Enter a binary string :");
    gets(input);
    char ch = input[i];
    while (ch != '\0') {
        curr_state = delta(curr_state, ch);
        ch = input[++i];
    }
    if (curr_state == qf) {
        printf("provided string is Accepted");
    } else {
        printf("provided string is Rejected");
    }
    return 0;
}

enum states delta( enum states s, char ch ) {
    enum states curr_state;
```

```

switch (s) {
    case q0:
        if (ch == '1')
            curr_state = q1;
        else
            curr_state = q0;
        break;
    case q1:
        if (ch == '0')
            curr_state = q2;
        else
            curr_state = q1;
        break;
    case q2:
        if (ch == '1')
            curr_state = q3;
        else
            curr_state = q0;
        break;
    case q3:
        if (ch == '1')
            curr_state = qf;
        else
            curr_state = q2;
        break;
    case qf:
        curr_state = qf;
        break;
    case qd:
        curr_state = qd;
        break;
}
return curr_state;
}

```

OUTPUT :-

Enter a binary string : 1010101011
provided string is Accepted.

Design a DFA that accepts strings of 0 & 1 which contains either 00 or 11 as substrings. also implement it.

```
> #include <stdio.h>
enum states {q0, q1, q2, q3, q4};
enum states delta (enum states, char);

int main() {
    char input [100];
    enum states curr_state = q0;
    int i = 0;
    printf("\n Enter a binary string : ");
    gets(input);
    char ch = input[i];
    while (ch != '\0') {
        curr_state = delta(curr_state, ch);
        ch = input[++i];
    }

    if (curr_state == q3 || curr_state == q4) {
        printf("provided string is Accepted\n");
    } else {
        printf("provided string is Rejected\n");
    }
    return 0;
}

enum states delta (enum states s, char ch) {
    enum states curr_state;
    switch (s) {
```


case q0:

```
if (ch == '0')  
    curr_state = q1;  
else  
    curr_state = q2;  
break;
```

case q1:

```
if (ch == '0')  
    curr_state = q3;  
else  
    curr_state = q2;  
break;
```

case q2:

```
if (ch == '1')  
    curr_state = q4;  
else  
    curr_state = q1;  
break;
```

case q3:

case q4:

```
curr_state = 5;  
break;
```

```
}  
return curr_state;
```

```
}
```

OUTPUT:

Enter a binary string: 110100100010
provided string is Accepted,

1> DFA for regular expression $r = (01+010)^*$

→ # include <stdio.h>

```
enum states { q0, q1, q2, q3 };
```

```
enum states delta (enum states, char);
```

```

int main() {
    char Input[100];
    enum states curr_state = q0;
    int i = 0;
    printf("Enter a binary string:");
    gets(Input);
    char ch = Input[i];
    while (ch != '\0') {
        curr_state = delta(curr_state, ch);
        ch = Input[++i];
    }
    if (curr_state == q0 || curr_state == q2 || curr_state == q3) {
        printf("Provided string is Accepted\n");
    } else {
        printf("Provided string is Rejected\n");
    }
    return 0;
}

enum states delta (enum states s, char ch) {
    enum states curr_state;
    switch (s) {
        case q0:
            if (ch == '0')
                curr_state = q1;
            else
                curr_state = q0;
            break;
        case q1:
            if (ch == '0')
                curr_state = q3;
            else
                curr_state = q2;
            break;
    }
}

```

case q2:

if (ch == '0')

curr_state = q1;

else

curr_state = q0;

break;

case q3:

if (ch == '0')

curr_state = q1;

else

curr_state = q0;

break;

}

return curr_state;

}

OUTPUT:

Enter a binary string: 11110000
provided string is rejected.

→ WAP to implement generic DFA which can simulate the any DFA defined by user.

→ #include <stdio.h>

#define MAX_STATES 100

#define MAX_ALPHABET 10

int main()

int num_states, num_alphabet, num_accepting_states;

int initial_state, accepting_states[MAX_STATES];

int transition[MAX_STATES][MAX_ALPHABET];

char alphabet[MAX_ALPHABET], input[100];

int curr_state;

//function to check if a state is accepting

int is-accepting-state (int s, int num_accepting_states,

```

int accepting_states[] {
    for (int i = 0; i < num_accepting_states; i++) {
        if (accepting_states[i] == s) {
            return 1;
        }
    }
    return 0;
}

```

// function to get the index of the alphabet symbol

```

int get_alphabet_index(char symbol, int num_alphabet,
    char alphabet[]) {
    for (int i = 0; i < num_alphabet; i++) {
        if (alphabet[i] == symbol) {
            return i;
        }
    }
    return -1; // Invalid symbol
}

```

```

// Read the number of states
printf("Enter the number of states (max %d):",
    MAX_STATES);
scanf("%d", &num_states);
if (num_states <= 0 || num_states > MAX_STATES) {
    printf("Error: Number of states must be between 1
        and %d.\n", MAX_STATES);
    return 1;
}
// Read the number of alphabet symbols
printf("Enter the number of alphabet symbols
    (max %d):", MAX_ALPHABET);

```



```

scanf ("%d", &num_alphabet);
if (num_alphabet <= 0 || num_alphabet > MAX_ALPHABET)
{ printf ("Error: Number of alphabet symbols must be
between 1 & %d. \n", MAX_ALPHABET);
return 1;
}
// Read the alphabet symbols
printf ("Enter the alphabet symbols:");
for (int i = 0; i < num_alphabet; i++) {
scanf ("%c", &alphabet[i]);
}

```

Transition table

```

printf ("Enter the transition table (use state numbers
starting from 0): \n");
for (int i = 0; i < num_states; i++) {
for (int j = 0; j < num_alphabet; j++) {
printf ("8 (q %d, %c) = q", i, alphabet[j]);
scanf ("%d", &transition[i][j]);
if (transition[i][j] < 0 || transition[i][j] >= num_states)
{ printf ("Error: Transition state must be between
0 and %d. \n", num_states - 1);
return 1;
}
}
}
}

```

```

// Read the initial state
printf ("Enter the initial state: q");
scanf ("%d", &initial_state);

```

```

if (initial_state < 0 || initial_state >= num_states) {
    printf("Error: Initial state must be between 0 and %d.\n", num_states - 1);
    return 1;
}

```

```

// Read the number of accepting states
printf("Enter the number of accepting states:");
scanf("%d", &num_accepting_states);
if (num_accepting_states < 0 || num_accepting_states > num_states) {
    printf("Error: Number of accepting states must be between 0 and %d.\n", num_states);
    return 1;
}

```

```

// Read the accepting states
printf("Enter the accepting states:");
for (int i = 0; i < num_accepting_states; i++) {
    scanf("%d", &accepting_states[i]);
    if (accepting_states[i] < 0 || accepting_states[i] >= num_states) {
        printf("Error: Accepting state must be between 0 and %d.\n", num_states - 1);
        return 1;
    }
}

```

```

// Enter a string to simulate (max 99 characters)
printf("Enter a string to simulate (max 99 characters):");
scanf("%s", input);
curr_state = initial_state;
for (int i = 0; input[i] != '\0'; i++) {
    int symbol_index = get_alphabet_index(input[i],

```

```

num_alphabet, alphabet );
if (symbol_index == -1) {
    printf("Invalid symbol %c in input \n", input[i]);
    return 1;
}
curr_state = transition[curr_state][symbol_index];
}
if (is_accepting_state(curr_state, num_accepting -
states_accepting_states)) {
    printf("provided string is Accepted \n");
} else {
    printf("provided string is Rejected \n");
}
return 0;
}

```

OUTPUT :-

Enter the number of states (max 100) : 3
Enter the number of alphabet symbols (max 10) : 2
Enter the alphabet symbols : 0 1
Enter the transition table (use state numbers
starting from 0) :
 $(q_0, 0) = q_1$
 $(q_0, 1) = q_0$
 $(q_1, 0) = q_2$
 $(q_1, 1) = q_0$
 $(q_2, 0) = q_2$
 $(q_2, 1) = q_2$
Enter the initial state : q_0
Enter the number of accepting states : 2
Enter the accepting states : 1 2

Enter a string to simulate (max 99 characters):
010
provided string is accepted.

DISCUSSION:

→ DFA are fundamental concepts in automata theory, providing a robust framework for understanding state machines & pattern recognition. The practical implementations provided in this lab demonstrate how DFAs can be designed and coded to recognize specific patterns in binary strings. Each program showcases different DFA designs, from simple patterns like '01' to more complex ones involving regular expressions and generic DFA simulations.

The implementation of DFAs reinforces the theoretical understanding of state transitions & their applications in real world scenarios. Understanding DFAs is crucial for fields such as compiler design, text processing and network protocol analysis, where pattern recognition and state transitions play a vital role.

CONCLUSION:

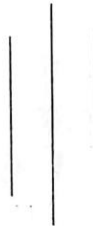
→ The detailed implementation of DFAs in a language illustrates the fundamental concepts of finite automata & their practical applications. The programs provided offer a comprehensive understanding of DFA design and simulation, reinforcing theoretical knowledge with practical coding experience. By experimenting with different DFA designs, students can gain valuable insights into the mechanics of state machines & their applications in various domains.



TRIBHUVAN UNIVERSITY
INSTITUTE OF SCIENCE AND TECHNOLOGY



HIMALAYA COLLEGE OF ENGINEERING
CHYASAL, LALITPUR



checked
[Signature]

Lab Report No:- 3

Title:- Implementation of NFA.

Submitted by:-

Name:- Surajan Shrestha.

Roll No:- 38

Submitted To:- Hari Prasad Pokharel

Department Of CSIT

Checked by:-

Date of submission:- 2081/04/23

TITLE - Implementation of NFA.

OBJECTIVE :-

→ To Implement Non-Deterministic finite Automaton (NFA) over the alphabet $(0,1)$ for different conditions using the C programming language.

THEORY :-

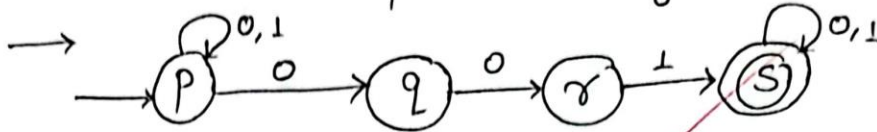
→ A NFA is a theoretical model of computation used in the study of computer science and automata theory. An NFA is defined by a set of states, a set of input symbols, a transition function, an initial state, and a set of accepting states. Unlike Deterministic finite Automata (DFA), an NFA can transition to multiple states from a given state on a particular input symbol. This non-determinism provides flexibility and is used to represent the regular languages.

An NFA is defined by the 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where :

- Q : finite set of states
- Σ : finite set of input symbols (alphabet)
- δ : Transition function mapping $Q \times \Sigma$ to the power set of Q
- q_0 : Initial state
- F : set of accepting states

OBSERVATIONS :

Activity - 1 : Implement NFA over alphabets $\{0,1\}$ that accepts substring "01".



source_code :

```
→ #include <stdio.h>
#include <string.h>
char input[20];
int l;
int flag;
void s(int i)
{
    flag = 1;
}
void r(int i)
{
    if (i < l) {
        if (input[i] == '1')
        {
            ++i;
            s(i);
        }
    }
}
void q(int i)
{
    if (i < l)
    {
        int k = i;
        if (input[k] == '0')
        {
            ++i;
            r(i);
        }
    }
}
}
```

```

void p(int i)
{
    if (i < 2)
    {
        int k = i;
        if (input[k] == '0')
        {
            k++;
            p(k);
            q(k);
        }
        else {
            if (input[i] == '1')
            {
                i++;
                p(i);
            }
        }
    }
}

```

```

int main()
{
    printf("\n Enter input string :");
    gets(input);
    l = strlen(input);
    int i = 0;
    flag = 0;
    p(i);
    if (flag == 1)
        printf("\n The string is accepted ---");
    else
        printf("\n The string is rejected ---");
    return 0;
}

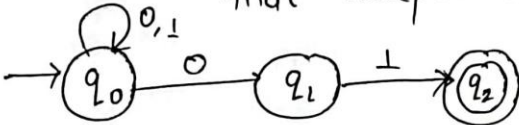
```


Output:

Enter input string : 011100001

The string is accepted ----

Activity-2 : Implement NFA over alphabet {0,1} that accepts strings ending with "01".



source_code:

```
→ #include <stdio.h>
#include <string.h>
char Input[20];
int l;
int flag;
void s(int i) {
    if (i==l) {
        flag = 1;
    }
}
void q(int i) {
    if (i<l) {
        if (Input[i]=='1') { s(i+1); }
        else { q(i+1); }
    }
}
void p(int i) {
    if (i<l) {
        if (Input[i]=='0') {
            q(i+1);
        }
    }
}
```

```

else
    p(i+1);
}
}

int main()
{
    printf("Enter input strings:");
    fgets(input, sizeof(input), stdin);
    l = strlen(input);
    if (l > 0 && input[l-1] == '\n')
        input[l-1] = '\0';
    l--;
}

flag = 0;
p(0);
if (flag == 1)
    printf("The string is accepted.\n");
else
    printf("The string is rejected.\n");
return 0;
}

```

INPUT:

Enter input string: 0101
 The string is accepted.

Activity-3: NFA over $\{0,1\}$ that accepts strings starting with '10' and ending with '1'.



Source_code :

```
→ #include <stdio.h>
#include <string.h>
char input[20];
int l;
int flag;
void s(int i){
    flag = 1;
}
void q(int i){
    if(i < l){
        if(input[i] == '1'){
            s(i+1);
        } else {
            q(i+1);
        }
    }
}
void p(int i){
    if(i < l){
        if(input[i] == '0'){
            q(i+1);
        } else {
            p(i+1);
        }
    }
}
int main(){
    printf("Enter input string: ");
    fgets(input, sizeof(input), stdin);
    l = strlen(input);
```

```

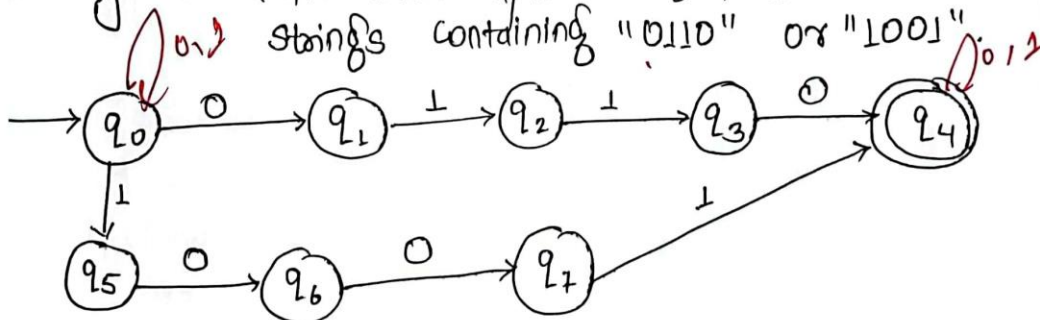
if (l > 0 && input[l-1] == '\n') {
    input[l-1] = '\0';
    l--;
}
flag = 0;
p(0);
if (flag == 1)
    printf("The string is accepted.\n");
else
    printf("The string is rejected.\n");
return 0;
}

```

OUTPUT :

Enter input string : 0100101
The string is accepted.

Activity - 4 : NFA over alphabets {0,1} that accepts strings containing "0110" or "1001"



Source_code :

```

#include <stdio.h> ✓
#include <string.h>
char input[20];
int l;
int flag;

```


// States for "0110" substring

```
void q3_0110(int p);  
void q2_0110(int p);  
void q1_0110(int p);  
void q0_0110(int p);
```

// States for "1001" substring

```
void q3_1001(int p);  
void q2_1001(int p);  
void q1_1001(int p);  
void q0_1001(int p);
```

void q3_0110(int p) {

if (input[p] == '0') {

flag = 1;

}

}

void q2_0110(int p) {

if (input[p] == '1') {

q3_0110(p+1);

}

}

void q1_0110(int p) {

if (input[p] == '1') {

q2_0110(p+1);

}

}

void q0_0110(int p) {

if (input[p] == '0') {

q1_0110(p+1);

}

}

```

void q3_1001(int i) {
    if (input[i] == '1') {
        flag = 1;
    }
}

```

```

}
void q2_1001(int i) {
    if (input[i] == '0') {
        q3_1001(i+1);
    }
}

```

```

}
void q1_1001(int i) {
    if (input[i] == '0') {
        q2_1001(i+1);
    }
}

```

```

}
void q0_1001(int i) {
    if (input[i] == '1') {
        q1_1001(i+1);
    }
}

```

```

}
void start(int i) {
    if (i < 2) {
        q0_0110(i);
        q0_1001(i);
        if (flag == 0) {
            start(i+1);
        }
    }
}

```

```

}
int main() {
    printf("Enter a binary string:");
    scanf("%s", input);
}

```

```

l = strlen(input);
flag = 0;
start(0);
if (flag == 1) {
    printf("The string is accepted.\n");
} else { printf("The string is rejected.\n");
}
return 0;
}

```

PUT :

Enter a binary string : 000011001001
 The string is accepted.

Discussion :

The implementation of NFAs in C effectively demonstrates the handling of non-deterministic state transitions through recursion function calls. Each NFA focuses on a specific string condition, showcasing their versatility. The first NFA accepts strings containing the substring "01", successfully identifying the sequence within the input. The second NFA ensures the strings ending with "01", validating the final characters. The third NFA targets strings that start with '0' and end with '1', irrespective of the intermediate characters. The fourth NFA checks for the presence of "010" or "1001" substrings, using multiple state functions.

recursive calls in these implementations simulate multiple state transitions accurately. Testing different input strings confirmed the correct behavior of each NFA. These NFAs illustrate how pattern recognition can be implemented using non-deterministic approaches. The use of the recursive functions ensures that all potential transitions are explored. Overall, the implementations highlight the power & flexibility of NFAs in string pattern recognition.

Conclusion :

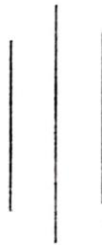
→ The lab exercise successfully implemented the various NFAs over the alphabet $\{0,1\}^*$. The recursive approach in C effectively simulated the non-deterministic behavior of NFAs. The results validated the correctness of each NFA for different string conditions. This exercise provided practical insights into NFA implementation & functioning. Key takeaways include understanding NFAs and their application in pattern recognition. The exercise reinforced the theoretical concepts of non-deterministic finite automata.



TRIBHUVAN UNIVERSITY
INSTITUTE OF SCIENCE AND TECHNOLOGY



HIMALAYA COLLEGE OF ENGINEERING
CHYASAL, LALITPUR



Checked
[Signature]

Lab Report No:- 4

Title:- Checking keywords, Identifiers and the
Comments in C

Submitted by:-

Name:- Surajan Shrestha

Roll No:- 38

Submitted To:- Hari Prasad Pochharel

Department Of CSIT

Checked by:-

Date of submission:- 2081/06/04

TITLE: checking keywords, Identifiers and the Comments in C.

OBJECTIVE:

- > The objective of this lab is to implement two programs - to:
 1. Detect keywords and identifiers in a given Input string using C programming.
 2. DETECT single-line and multi-line comments in C code.

THEORY:

→ In programming, keywords are reserved words that have special meanings in a language and cannot be used for other purposes like variable names or functions. Identifiers are names given to various program elements such as variables, functions, arrays, etc. A valid identifier in C must begin with a letter or underscore and can consist of alphanumeric characters or the underscores. This lab also covers the detection of comments, which are non-executable lines in code intended for documentation or explanation.

Keywords:

→ Keywords are predefined and reserved words that are recognized by the compiler. Example include int, return, if and else. These cannot be used as identifiers (names for variables, functions, etc).

Identifiers:

→ Identifiers are user-defined names that represent variables, functions, arrays, etc. An identifier must start with a letter or underscore, followed by letters or digits. For example, valid identifiers are `myvar`, `_values1`, and `function_name`.

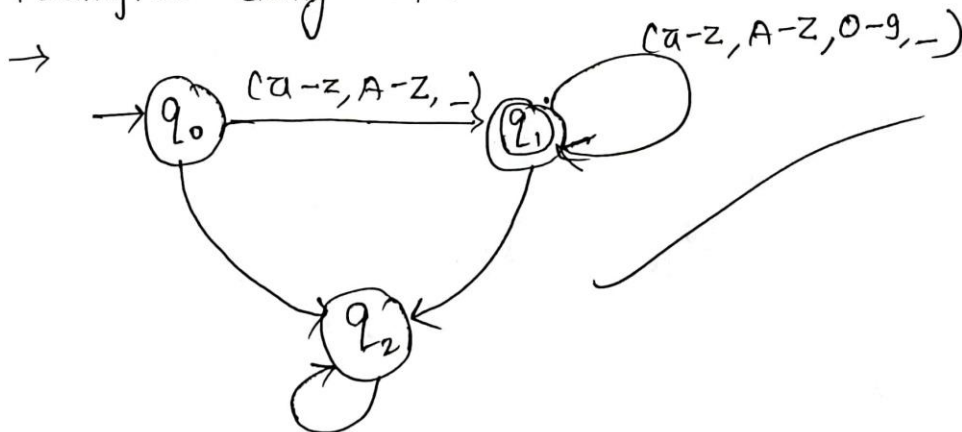
Comments:

→ Comments are used for code documentation & are ignored by the compiler. C supports two types of comments:

- single-line comments starting with `//`.
- multi-line comments enclosed between `/*` and `*/`.

OBSERVATION:

Activity 1: WAP to check either the given string is keyword or not. If the given string is not a keyword then check if it is valid C identifier using DFA.



```

#include <stdio.h>
#include <string.h>
#include <ctype.h>
// list of C keywords
const char * keywords[] = {
    "auto", "break", "case", "char", "const", "continue", "default",
    "do", "double", "else", "enum", "extern", "float", "for", "goto",
    "if", "int", "long", "register", "return", "short", "signed",
    "void", "volatile", "while", "sizeof" };

int iskeyword(char * str) {
    for (int i = 0; i < sizeof(keywords) / sizeof(char *); i++) {
        if (strcmp(str, keywords[i]) == 0) {
            return 1;
        }
    }
    return 0;
}

int isValidIdentifier(char * str) {
    if (!isalpha(str[0]) && str[0] != '_') {
        return 0;
    }
    for (int i = 1; i < strlen(str); i++) {
        if (!isalnum(str[i]) && str[i] != '_') {
            return 0;
        }
    }
    return 1;
}

```



```

int main() {
    char str[100];
    printf("Enter a string:");
    scanf("%s", str);
    if (iskeyword(str)) {
        printf("%s' is a keyword in c.\n", str);
    }
    else if (isvalidIdentifier(str)) {
        printf("' %s' is a valid identifier.\n", str);
    }
    else {
        printf("' %s' is not a valid identifier.\n", str);
    }
    return 0;
}

```

Output 1:

Enter a string: int
 'int' is a keyword in c.

Output 2:

Enter a string: my_variable
 'my_variable' is a valid identifier.

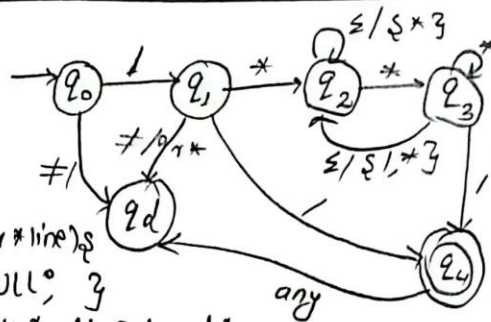
Activity - 2:

Design DFA and wap to check either given string is comment or not. Your program should check single as well as multi line comment.

```
# Include <stdio.h>
#include <string.h>
#include <stdbool.h>
#define N 1024
```

```
bool is_single_line_cmt(const char *line) {
    return strstr(line, "//") != NULL;
}

bool is_multiline_cmt(const char *line, bool *in_cmt) {
    const char *start = "/*";
    const char *end = "*/";
    if (*in_cmt) {
        if (strstr(line, end) != NULL) {
            *in_cmt = false;
            return true;
        }
        return true;
    }
}
```



```
if (strstr(line, start) != NULL) {
    if (strstr(line, end) != NULL) {
        return true;
    }
    *in_cmt = true;
    return true;
}
return false;
```

```
int main() {
    char line[N];
    bool in_cmt = false;
    printf("Enter lines of text : \n");
    while (fgets(line, sizeof(line), stdin)) {
        line[strcspn(line, "\n")] = '\0';
        if (is_single_line_cmt(line)) {
            printf("line : '%s' - single-line cmt \n", line);
        } else if (is_multiline_cmt(line, &in_cmt)) {
            if (in_cmt) {
                printf("line : '%s' - multi-line cmt (in-progress) \n", line);
            } else {
                printf("line : '%s' - multi-line cmt \n", line);
            }
        } else {
            printf("line : '%s' - Not a cmt \n", line);
        }
        if (!in_cmt) {
            printf("Warning: multi-line cmt not closed \n");
        }
    }
    return 0;
}
```

/*
Enter a comment : // This is single-line comment
This is a valid comment.

DISCUSSION :

→ The lab experiment focused on identifying C keywords, valid identifiers and detecting comments using DFA. The implementation of DFA helped to determine whether a string adheres to syntactical rules of C identifiers, while a separate DFA validated single-line and multi-line comments. By using state transitions based on character input, both DFA's effectively distinguished between valid and invalid inputs.

CONCLUSION :

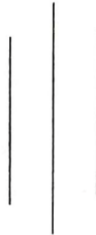
→ The experiment demonstrated the practical application of DFA in lexical analysis by successfully checking C keywords, identifiers and comments. DFA serves as an essential tool in compilers for ensuring syntactical correctness, enhancing error detection during the initial stages of code compilation.



TRIBHUVAN UNIVERSITY
INSTITUTE OF SCIENCE AND TECHNOLOGY



HIMALAYA COLLEGE OF ENGINEERING
CHYASAL, LALITPUR



Lab Report No:- 05

Title:- Implementation of PDA

Submitted by:-

Name:- Swaraj Shrestha

Roll No:- 38

Submitted To:- Hari Prasad Pokharel

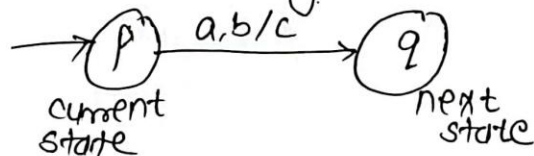
Department Of CSIT

Checked by:-

Date of submission:- 2081/06/08

$c \rightarrow$ symbol to be push into top of stack or pop the top of stack symbol.

Transition diagram:



a = input symbol
 b = top of stack symbol
 c = symbol to be push/pop.

OBSERVATION:

1. Design a PDA that accepts all the strings over $\{0,1\}$ that have any number of 0's followed by same no. of 1's i.e. $0^n 1^n$, $n > 0$ by final state. Implement it in C.

```
#include <stdio.h>
```

```
#define MAX 100
```

```
enum state {q0, q1, q2};
```

```
void push(char ch);
```

```
char get_stack_top();
```

```
void pop();
```

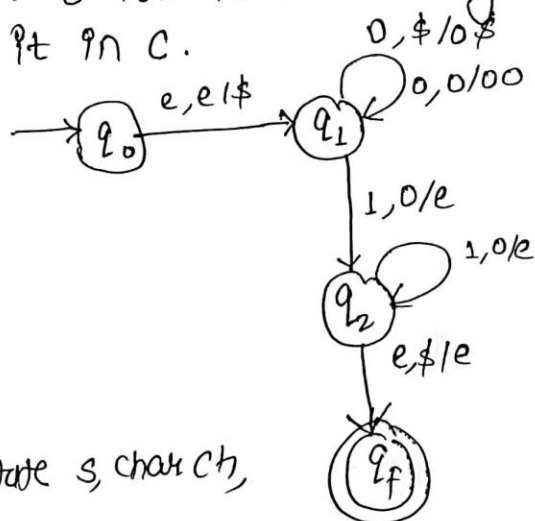
```
enum state delta(enum state s, char ch,  
char st_top);
```

```
Stack stack;
```

```
char symbols[MAX];
```

```
int top;
```

```
int st;
```



```

void push(char ch) {
    if (st.top < MAX - 1) {
        st.symbols[++st.top] = ch;
    } else {
        printf("stack full\n");
    }
}

```

```

char get_stack_top() {
    if (st.top > -1) {
        return st.symbols[st.top];
    }
    return '\0';
}

```

```

void pop() {
    if (st.top > -1) {
        st.symbols[st.top--] = '\0';
    } else {
        printf("stack Empty\n");
    }
}

```

```

enum state delta (enum state s, char ch, char st_top) {
    switch (s) {
        case q0:
            if (ch == '0' && st_top == 'e') {
                push('x');
                return q0;
            } else if (ch == '0' && st_top == 'x') {
                push('x');
                return q0;
            }
    }
}

```

```

else if (ch == '1' && st_top == 'x') {
    pop();
    return q1;
}
break;
case q1:
    if (ch == '1' && st_top == 'x') {
        pop();
        return q1;
    }
    break;
}
return q2;

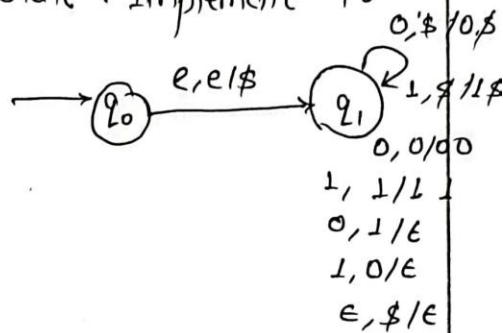
```

```

int main() {
    st.top = -1;
    char input[20];
    int state curr_state = q0;
    printf("Enter a binary string:");
    scanf("%s", input);
    scanf("%c", &ch);
    int i = 0;
    while (input[i] != '\0') {
        char st_top = get_stack_top();
        curr_state = delta(curr_state, input[i], st_top);
        i++;
    }
    if (curr_state == q1 && get_stack_top() == 'e') {
        printf("The string %s is accepted\n", input);
    } else {
        printf("The string %s is rejected\n", input);
    }
    return 0;
}

```

Q.2, a, Design a PDA that accepts all the strings over $\{0,1\}$ that have equal no. of 0's & 1's by Empty state. Implement it using C.



```

#include <stdio.h>
#include <string.h>
#define MAX 100
enum states {q0, q1};

```

```

struct stack {
    char symbols[MAX];
    int top;
};

```

```

void push(struct stack *s, char ch) {

```

```

    if (s->top < MAX - 1) {
        s->symbols[++(s->top)] = ch;
    }

```

```

}

char pop(struct stack *s) {

```

```

    if (s->top >= 0) {
        return s->symbols[(s->top)--];
    }
    return '\0';
}

```

```

char get_stack_top(struct stack *s) {

```

```

    if (s->top >= 0) {
        return s->symbols[s->top];
    }

```

```

    return '\0';
}

```

```

enum states delta(enum states curr_state, char ch,
    char st_top) {

```



```

if (curr_state == q0) {
    if (ch == '0') {
        return q0;
    } else if (ch == '1' && st_top == '0') {
        return q0;
    }
}
return curr_state;

```

```

int main() {
    char input[20];
    enum states curr_state = q0;
    struct stack s;
    s.top = -1;
    printf("Enter a binary string:");
    gets(input);
    int i = 0;
    char ch = '\0';
    while (input[i] != '\0') {
        ch = input[i];
        char st_top = get_stack_top(&s);
        curr_state = delta(curr_state, ch, st_top);
        if (ch == '0') {
            push(&s, '0');
        } else if (ch == '1') {
            pop(&s);
        }
        i++;
    }
    if (s.top == -1) {
        printf("string accepted by empty stack\n");
    } else {
        printf("string rejected\n");
    }
}

```

return 0;

}

OUTPUT:

Enter a binary string: 0101
string accepted by empty stack.

→ design a pda that accepts all string over $\{0,1\}^*$ that have equal no of 0's and 1 by final state.

```
#include <stdio.h>
#include <string.h>
#define MAX 100
enum states {q0, qf};
struct stack {
    char symbols[MAX];
    int top;
```

};

```
void push (struct stack *s, char ch) {
    if (s->top < MAX-1) {
        s->symbols[++(s->top)] = ch;
```

}

```
void pop (struct stack *s) {
```

```
    if (s->top >= 0) {
```

```
        return s->symbols[(s->top)--];
```

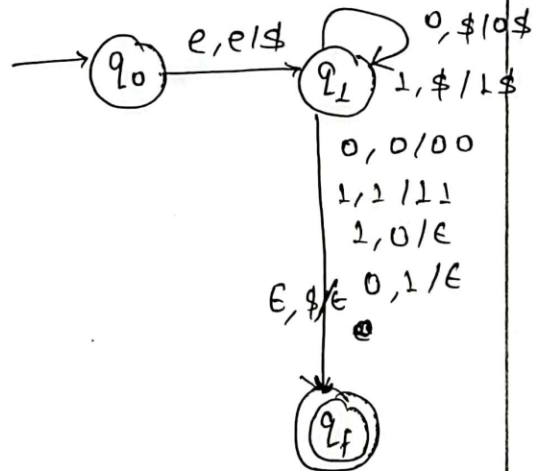
```
    }
```

```
    return '\0';
```

```
}
```

```
char get_stack_top (struct stack *s) {
```

```
    if (s->top >= 0) return s->symbols[s->top];
```



```

    }
    return '10';
}

enum states delta (enum states cur_state, char ch,
char st_top, struct stack* s) {
    if (cur_state == q0) {
        if (ch == '0') {
            push(s, '0');
            return q0;
        } else if (ch == '1' && st_top == '0') {
            pop(s);
            return q0;
        }
    }
    if (cur_state == q0 && ch == '10' && s->top == 1) {
        return qf;
    }
    return cur_state;
}

```

```

int main() {
    char input[20];
    enum states cur_state = q0;
    struct stack s;
    s.top = -1;
    printf("Enter a binary string: ");
    gets(input);
    int i = 0;
    char ch;
    while ((ch = input[i]) != '\0') {
        char st_top = get_stack_top(&s);
        cur_state = delta(cur_state, ch, st_top, &s);
        i++;
    }
}

```

```

curr_state = delta(curr_state, '\0', get_stack_top(&s)
& s);
if (curr_state == qf) {
    printf("string accepted by final state \n");
} else {
    printf("string rejected \n");
}
return 0;
}

```

OUTPUT:

Enter a binary string: 0101
 string accepted by final state.

3. Design a PDA that accepts strings in the form $wcwr$, where w is any string over $\{0,1\}$, and wr is reverse of w .

```

#include <stdio.h>
#include <string.h>
#define MAX 100
enum states {q0, q1, qf};
struct stack {
    char symbols[MAX];
    int top;
} st;

void push(struct stack *s, char ch) {
    if (s->top < MAX - 1) {
        s->symbols[++(s->top)] = ch;
    } else {
        printf("stack full \n");
    }
}

```



```

char pop(struct stack *s) {
    if (s->top >= 0) {
        return s->symbols[(s->top)--];
    } else {
        printf("stack Empty \n");
        return '\0';
    }
}

```

```

char get_stack_top(struct stack *s) {
    if (s->top >= 0) {
        return s->symbols[s->top];
    }
    return '\0';
}

```

```

enum states delta (enum states curr_state, char ch,
struct stack *s, int len, int p) {
    if (curr_state == q0) {
        if (p < len/2) {
            push(s, ch);
            return q0;
        } else {
            return q1;
        }
    } else if (curr_state == q1) {
        char st_top = get_stack_top(s);
        if (ch == st_top) {
            pop(s);
            return q1;
        } else {
            return curr_state;
        }
    }
}

```

```

}
}
if (curr_state == q1 && s -> top == -1) {
    return qf;
}
return curr_state;
}

int main() {
    st3.top = -1;
    char input[20];
    printf("Enter a binary string");
    scanf("%s", input);
    int len = strlen(input);
    int i = 0;
    states curr_state = q0;
    while (i < len) {
        curr_state = delta(curr_state, input[i], &st3, len, i);
        i++;
    }
    curr_state = delta(curr_state, '10', &st3, len, i);
    if (curr_state == qf)
    {
        printf("The string %s is accepted\n", input);
    }
    else {
        printf("The string %s is rejected\n", input);
    }
}

```

return 0;

}

OUTPUT:

Enter a binary string: 01010
The string 01010 is accepted.

DISCUSSION:

→ In this lab, we implemented three PDA's using C to recognize specific language patterns. Activity 1 designed a PDA to accept strings where the number of 0's is followed by the same number of 1's, using a stack to match counts. Activity 2 implemented a PDA that accepts strings with equal number of 0's and 1's, regardless of order, by tracking the balance using the stack. Activity 3 focused on palindromes, using the stack to store the first half of the string and compare it with the reverse. The use of C for stack operations allowed efficient memory management and correct state transitions, validating the practical application of PDA's in language recognition tasks.

CONCLUSION:

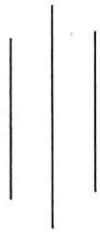
→ The implementation of PDAs demonstrated their effectiveness in handling context free languages by recognizing specific patterns like equal number of 0's & 1's and palindromes. The stack was crucial in managing state transitions and intermediate symbols. Overall, the lab reinforced PDA concepts and validated their practical use in language recognition with manageable memory constraints.



TRIBHUVAN UNIVERSITY
INSTITUTE OF SCIENCE AND TECHNOLOGY



HIMALAYA COLLEGE OF ENGINEERING
CHYASAL, LALITPUR



Checked
8

Lab Report No:- 5

Title:- Implementation of Turing machine

Submitted by:-

Name:- Swaraj Shrestha

Roll No:- 38

Submitted To:- Hari prasad pokharel

Department Of CSIT

Checked by:-

Date of submission:- 2081/06/08

TITLE: Implementation of Turing machine

OBJECTIVE:

- ↳ • To implement Turing machines for the following three tasks.
 - a, Recognizing the language $L = \{0^n 1^n \mid n \geq 1\}$, where the number of 0's is equal to the number of 1's.
 - b, Complementing a binary string.
 - c, Performing binary addition on 2 binary numbers.
- To explore the Turing machine's functioning, highlighting how it handles both language recognition and computational tasks.

THEORY:

↳ A Turing machine (TM) is a theoretical computational model that manipulates symbols on a tape according to a set of rules. It consists of:

- Tape = A potentially infinite memory storage where input is placed and can be read/written.
- Head = A pointer that reads or writes symbols on the tape.
- States = The machine can be in one of a finite number of states. It changes states based on the current input symbol and transition rules.
- Alphabet = A set of symbols, including a blank symbol, used for computation.
- Transition function = Defines the machine's action when

2. a certain state reading a specific symbol, determining the next state, tape movement and what to write.

1. In this lab, we design and simulate three different Turing machines:

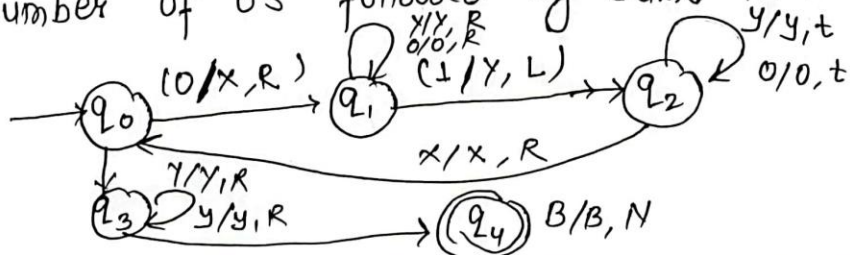
• Tm for $L = \{0^n 1^n\}$: This machine accepts the binary strings with n 0's followed by n 1's ensuring that the count of 0's matches the count of 1's.

• Tm for Binary string complement: This machine reads a binary string and replaces 0's with 1's and 1's with 0's, thereby complementing the string.

• Tm for Binary Addition: This machine adds two binary numbers bit by bit, performing the addition using rules similar to the manual binary addition with carry handling.

Observation:

Q.1, Implement a Tm for $L = \{ \text{all strings over } \{0,1\} \text{ such that the string has any number of 0's followed by same number of 1's} \}$.



```

// Turing machine for  $L = \{0^n 1^n\}$ 
#include <stdio.h>
#include <string.h>
int isLanguageAccepted(char *str) {
    int len = strlen(str);
    int i = 0;
    while (i < len && str[i] == '0') i++;
    int numZeros = i;
    while (i < len && str[i] == '1') i++;
    int numOnes = i - numZeros;
    return (numZeros == numOnes && i == len);
}
int main() {
    char str[100];
    printf("Enter a binary string (0's followed by 1's):");
    scanf("%s", str);
    if (isLanguageAccepted(str)) {
        printf("The string '%s' is accepted by the Turing machine ( $L = 0^n 1^n$ ).\n", str);
    } else {
        printf("The string '%s' is not accepted by the Turing machine.\n", str);
    }
    return 0;
}

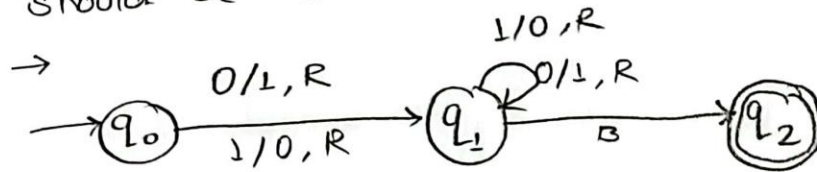
```

INPUT:

Enter a binary string (0's followed by 1's):
10110101010

The string '00110101010' is NOT accepted by the Turing machine.

2.2> Design a TM that perform the following function $f(x) = \text{complement of } x$. Also write to implement it. If input is 110 the output should be 001.



// Turing machine to complement a Binary string.

```
#include <stdio.h>
```

```
#include <string.h>
```

```
void complementBinaryString(char *str) {
```

```
    int len = strlen(str);
```

```
    for (int i = 0; i < len; i++) {
```

```
        if (str[i] == '0') {
```

```
            str[i] = '1';
```

```
        }
```

```
        else if (str[i] == '1') {
```

```
            str[i] = '0';
```

```
        }
```

```
    }
```

```
}
```



```

int main() {
    char str[100];
    printf("Enter a binary string : ");
    scanf("%s", str);
    complementBinaryString(str);
    printf("The complement of binary string is : %s\n", str);
    return 0;
}

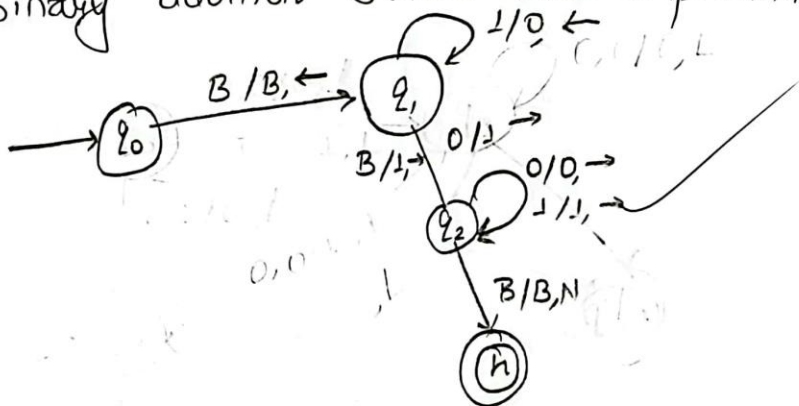
```

Output :

Enter a binary string : 110
 The complement of binary string is : 001.

Q.3. Design a Turing machine that perform binary addition tasks. Also implement it.

→



// Turing machine for Binary Addition

```
#include <stdio.h>
```

```
#include <string.h>
```

```

void binaryAddition (char* num1, char* num2, char* result)
{
    int i = strlen(num1) - 1;
    int j = strlen(num2) - 1;
    int carry = 0, k = 0;
    while (i >= 0 || j >= 0 || carry) {
        int bit1 = (i >= 0) ? num1[i] - '0' : 0;
        int bit2 = (j >= 0) ? num2[j] - '0' : 0;
        int sum = bit1 + bit2 + carry;
        result[k++] = (sum % 2) + '0';
        carry = sum / 2;
        i--;
        j--;
    }
    result[k] = '\0';

    for (int l = 0; l < k/2; l++) {
        char temp = result[l];
        result[l] = result[k-l-1];
        result[k-l-1] = temp;
    }

    int main() {
        char num1[100], num2[100], result[100];
        printf("Enter the first binary number : ");
        scanf("%s", num1);
        printf("Enter the second binary number : ");
        scanf("%s", num2);
        binaryAddition(num1, num2, result);
        printf("The result of binary addition is : %s\n", result);
    }
}

```

return 0;

}

OUTPUT:

Enter the first binary number: 1111

Enter the second binary number: 1001

The result of binary addition: 11000

Discussion :

→ The Turing machine in this lab highlights key aspects of computational theory. For the language $L = \{0^n 1^n\}$, it shows how formal languages can be processed through state transitions & tape operations. The complementing binary string machine demonstrates the machine's ability to handle data transformations, while the binary addition machine showcases its power in performing arithmetic, managing carry bits via state transitions. These implementations illustrate how the Turing machines, though abstract, can solve practical problems, making them foundational for understanding computation limits and algorithm design.

Conclusion:

→ The Turing machine designed in this lab successfully achieves the three tasks: recognizing the language $L = \{0^n 1^n\}$, complementing binary strings and performing binary addition. Each TM demonstrates the power and flexibility of this computational model in both theoretical and practical contexts. These exercises underscore the value of TM in formal language processing, algorithmic computations and theoretical computer science.